

Sample PDF

This is a simple PDF file. Fun fun fun.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Phasellus facilisis odio sed mi. Curabitur suscipit. Nullam vel nisi. Etiam semper ipsum ut lectus. Proin aliquam, erat eget pharetra commodo, eros mi condimentum quam, sed commodo justo quam ut velit. Integer a erat. Cras laoreet ligula cursus enim. Aenean scelerisque velit et tellus. Vestibulum dictum aliquet sem. Nulla facilisi. Vestibulum accumsan ante vitae elit. Nulla erat dolor, blandit in, rutrum quis, semper pulvinar, enim. Nullam varius congue risus. Vivamus sollicitudin, metus ut interdum eleifend, nisi tellus pellentesque elit, tristique accumsan eros quam et risus. Suspendisse libero odio, mattis sit amet, aliquet eget, hendrerit vel, nulla. Sed vitae augue. Aliquam erat volutpat. Aliquam feugiat vulputate nisl. Suspendisse quis nulla pretium ante pretium mollis. Proin velit ligula, sagittis at, egestas a, pulvinar quis, nisl.

Pellentesque sit amet lectus. Praesent pulvinar, nunc quis iaculis sagittis, justo quam lobortis tortor, sed vestibulum dui metus venenatis est. Nunc cursus ligula. Nulla facilisi. Phasellus ullamcorper consectetur ante. Duis tincidunt, urna id condimentum luctus, nibh ante vulputate sapien, id sagittis massa orci ut enim. Pellentesque vestibulum convallis sem. Nulla consequat quam ut nisl. Nullam est. Curabitur tincidunt dapibus lorem. Proin velit turpis, scelerisque sit amet, iaculis nec, rhoncus ac, ipsum. Phasellus lorem arcu, feugiat eu, gravida eu, consequat molestie, ipsum. Nullam vel est ut ipsum volutpat feugiat. Aenean pellentesque.

In mauris. Pellentesque dui nisi, iaculis eu, rhoncus in, venenatis ac, ante. Ut odio justo, scelerisque vel, facilisis non, commodo a, pede. Cras nec massa sit amet tortor volutpat varius. Donec lacinia, neque a luctus aliquet, pede massa imperdiet ante, at varius lorem pede sed sapien. Fusce erat nibh, aliquet in, eleifend eget, commodo eget, erat. Fusce consectetur. Cras risus tortor, porttitor nec, tristique sed, convallis semper, eros. Fusce vulputate ipsum a mauris. Phasellus mollis. Curabitur sed urna. Aliquam nec sapien non nibh pulvinar convallis. Vivamus facilisis augue quis quam. Proin cursus aliquet metus. Suspendisse lacinia. Nulla at tellus ac turpis eleifend scelerisque. Maecenas a pede vitae enim commodo interdum. Donec odio. Sed sollicitudin dui vitae justo.

Morbi elit nunc, facilisis a, mollis a, molestie at, lectus. Suspendisse eget mauris eu tellus molestie cursus. Duis ut magna at justo dignissim condimentum. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Vivamus varius. Ut sit amet diam suscipit mauris ornare aliquam. Sed varius. Duis arcu. Etiam tristique massa eget dui. Phasellus congue. Aenean est erat, tincidunt eget, venenatis quis, commodo at, quam.

Welcome to pypdf

pypdf is a [free](#) and open source pure-python PDF library capable of splitting, merging, cropping, and transforming the pages of PDF files. It can also add custom data, viewing options, and passwords to PDF files. pypdf can retrieve text and metadata from PDFs as well.

See [pdfly](#) for a CLI application that uses pypdf to interact with PDFs.

You can contribute to [pypdf on GitHub](#).

User Guide

- [Installation](#)
- [Migration Guide: 1.x to 2.x](#)
- [Imports and Modules](#)
- [Naming Adjustments](#)
- [Robustness and strict=False](#)
- [Exceptions, Warnings, and Log messages](#)
- [Metadata](#)
- [Extract Text from a PDF](#)
- [Post-Processing of Text Extraction](#)
- [Extract Images](#)
- [Other images](#)
- [Extract Attachments](#)
- [Encryption and Decryption of PDFs](#)
- [Merging PDF files](#)
- [Cropping and Transforming PDFs](#)
- [Transforming several copies of the same page](#)
- [Reading PDF Annotations](#)
- [Adding PDF Annotations](#)
- [Adding a Stamp or Watermark to a PDF](#)
- [Adding JavaScript to a PDF](#)
- [Adding Viewer Preferences](#)
- [Interactions with PDF Forms](#)
- [Streaming Data with pypdf](#)
- [Reduce PDF File Size](#)
- [PDF Version Support](#)
- [PDF/A Compliance](#)

API Reference

- [The PdfReader Class](#)
- [The PdfWriter Class](#)
- [The Destination Class](#)

- [The DocumentInformation Class](#)
- [The Field Class](#)
- [The Fit Class](#)
- [The PageObject Class](#)
- [The PageRange Class](#)
- [The PaperSize Class](#)
- [The RectangleObject Class](#)
- [The Transformation Class](#)
- [The XmpInformation Class](#)
- [The annotations module](#)
- [Constants](#)
- [Errors](#)
- [Generic PDF objects](#)
- [The PdfDocCommon Class](#)

Developer Guide

- [Developer Intro](#)
- [The PDF Format](#)
- [How pypdf parses PDF files](#)
- [How pypdf writes PDF files](#)
- [CMaps](#)
- [The Deprecation Process](#)
- [Documentation](#)
- [Testing](#)
- [Releasing](#)

About pypdf

- [CHANGELOG](#)
- [Changelog of PyPDF2 1.X](#)
- [Project Governance](#)
- [Taking Ownership of pypdf](#)
- [History of pypdf](#)
- [Contributors](#)
- [Scope of pypdf](#)
- [pypdf vs X](#)
- [Frequently Asked Questions](#)

Indices and tables

- [Index](#)
- [Module Index](#)
- [Search Page](#)

Merging PDF files

Basic Example

```
from PyPDF2 import PdfWriter

merger = PdfWriter()

for pdf in ["file1.pdf", "file2.pdf", "file3.pdf"]:
    merger.append(pdf)

merger.write("merged-pdf.pdf")
merger.close()
```

For more details, see an excellent answer on [StackOverflow](#) by Paul Rooney.

Showing more merging options

```
from PyPDF2 import PdfWriter

merger = PdfWriter()

input1 = open("document1.pdf", "rb")
input2 = open("document2.pdf", "rb")
input3 = open("document3.pdf", "rb")

# add the first 3 pages of input1 document to output
merger.append(fileobj=input1, pages=(0, 3))

# insert the first page of input2 into the output beginning after the second page
merger.merge(position=2, fileobj=input2, pages=(0, 1))

# append entire input3 document to the end of the output document
merger.append(input3)

# Write to an output PDF document
output = open("document-output.pdf", "wb")
merger.write(output)

# Close File Descriptors
merger.close()
output.close()
```

append

`append` has been slightly extended in `PdfWriter`.

see [pdfWriter.append](#) for more details

parameters:

fileobj: PdfReader or filename to merge *outline_item*: string of a outline/bookmark pointing to the beginning of the inserted file. if None, or omitted, no bookmark will be added. *pages*: pages to merge ; you can also provide a list of pages to merge None(default) means that the full document will be merged. *import_outline*: import/ignore the pertinent outlines from the source (default True) *excluded_fields*: list of keys to be ignored for the imported objects; if “/Annots” is part of the list, the annotation will be ignored if “/B” is part of the list, the articles will be ignored

examples:

```
writer.append("source.pdf",(0,10)) # append the first 10 pages of source.pdf
```

```
writer.append(reader,"page 1 and 10",[0,9]) #append first and 10th page from reader and create an outline)
```

During the merging, the relevant named destination will also imported.

If you want to insert pages in the middle of the destination, use merge (which provides (insert) position)

You can now insert the same page multiple times. You can also insert the same page many time at once with a list:

eg: `writer.append(reader,[0,1,0,2,0])` will insert the pages (1), (2), with page (0) before, in the middle and after

add_page / insert_page

It is recommended to use `append` or `merge` instead

reset_translation

During the cloning, if an object has been already cloned, it will not be cloned again, a pointer this previously cloned object is returned. because of that, if you add/merge a page that has been already added, the same object will be added the second time. If later you modify any of these two page, both pages can be modified independantly.

To reset, call `writer.reset_translation(reader)`

Advanced cloning

In order to prevent side effect between pages/objects objects and all objects linked are linked during merging.

This process will be automatically applied if you use PdfWriter.append/merge/add_page/insert_page. If you want to clone an object before attaching it “manually”, use clone function of any PdfObject: eg:

```
cloned_object = object.clone(writer)
```

if you try clone an object already belonging to writer, it will return the same object

```
cloned_object == object.clone(writer) # -> returns True
```

the same, if you try to clone twice an object it will return the previously cloned object

```
object.clone(writer) == object.clone(writer) # -> returns True
```

Also, note that if you clone an object, you will clone all the objects below including the objects pointed by IndirectObject. because of that if you clone a page that includes some articles ("/B"), not only the first article, but also all the chained articles, and the pages where those articles can be read will be copied. It means that you may copy lots of objects, that will be saved in the output pdf.

In order to prevent, that you can provide the list of defined the fields in the dictionaries to be ignored:

eg:

```
new_page = writer.add_page(reader.pages[0],excluded_fields=["/B"])
```